

■ SESIÓN 9

Diseñar Tic Tac Toe sin código

Duración:	1 hora
Fase:	Abril - Juegos como módulos
Conceptos clave:	Lógica, estado, casos borde, pseudocódigo
Resultado esperado:	lógica del juego en pseudocódigo (sin JavaScript)

1. Idea Central de la Sesión

El Tic Tac Toe parece simple: dos jugadores, 9 casillas, gana quien alinea 3. Pero si lo observamos como ingeniera, vemos un sistema con **reglas, estado, transiciones y casos borde**. Esta sesión trata de entender esos mecanismos *antes de escribir una sola línea de código*.

Pregunta guía:

¿Cómo describirías el Tic Tac Toe a alguien que nunca lo ha visto, pero que entiende lógica y reglas? ¿Qué necesita saber el código para jugar?

2. Conexión con Conocimientos Previos

De Scratch:

En Scratch, probablemente hiciste juegos con si/si-no bloques. Pero todo estaba mezclado: dibujos, sonidos, lógica. Hoy aprendemos a **separar** la *lógica del juego* de su *visualización*.

De la app de juegos:

Ya sabes cómo navega la app (menú → juego → resultado) y cómo el estado recuerda en qué pantalla estamos. Ahora veremos cómo el **estado interno del juego** (tablero, turno, ganador) funciona con el mismo patrón: **evento** → **cambio de estado** → **UI se actualiza**.

De ingeniería:

Separación de responsabilidades (Sesión 7): la **lógica del juego** vive en un lugar, la **UI** en otro. El Tic Tac Toe es el ejemplo perfecto para ver esto.

3. ¿Qué es el Tic Tac Toe realmente?

Para el usuario es un juego: hace click, aparece su marca, la app responde. Pero para nosotras como ingenierass, el Tic Tac Toe es un **sistema de reglas** que transforma **entrada** (movimiento del usuario) en **salida** (nuevo estado del tablero, turno del rival, resultado).

Los elementos clave del juego:

Elemento	¿Qué es?	¿Quién lo controla?
Tablero	Una cuadrícula 3×3 con 9 casillas	El juego (estado)
Jugadores	2 jugadores (X y O)	El juego (turno)
Turno	Cuyo toca jugar ahora	El juego (lógica)
Movimiento	Elegir una casilla vacía	El usuario (evento)
Validación	¿Se puede jugar ahí?	El juego (reglas)
Victoria	¿Hay 3 en línea?	El juego (lógica)
Empate	¿Está lleno sin ganador?	El juego (lógica)

4. Diseño Antes de Código

Antes de escribir JavaScript, necesitamos responder estas preguntas:

- **¿Cómo representamos el tablero?**
→ ¿Array? ¿Objeto? ¿Qué datos guardar en cada casilla?
- **¿Cómo sabemos de quién es el turno?**
→ Una variable que alterna entre X y O
- **¿Cómo validamos un movimiento?**
→ ¿Existe ya una marca en esa casilla?
- **¿Cómo calculamos victoria?**
→ ¿Revisamos todas las líneas? ¿Hay un patrón?
- **¿Cómo empieza el juego?**
→ Tablero vacío, X siempre empieza
- **¿Cómo termina?**
→ Victoria o empate
- **¿Qué puede fallar?**
→ Casos especiales que la lógica debe prever

5. Cómo Encaja en la App

El Tic Tac Toe no existe solo. Es un módulo dentro de la app. Necesita comunicarse con el sistema general:

APP (estado global) ■■■ pantallaActual: "juego-tic-tac-toe" ■■■ puntuación: 5 ■■■ **juegoActivo: { estado del TicTacToe }** **TICTA CTOE (estado local del juego)** ■■■ tablero: [[], [], []] ■■■ turno: "X" o "O" ■■■ ganador: null ■■■ estado: "jugando" o "terminado"

Flujo de comunicación:

Usuario hace click en casilla ↓ App detecta evento "click" ↓ App llama función: juegoTicTacToe.hacerMovimiento(fila, col) ↓ Lógica del juego: • Valida el movimiento • Actualiza el tablero (estado) • Revisa si hay victoria • Cambia el turno ↓ App recibe el nuevo estado del juego ↓ App actualiza la UI (pantalla)

6. Casos Borde (Lo Que Puede Fallar)

Un buen diseño prevé qué puede ir mal. Aquí hay casos que nuestra lógica debe manejar:

- Usuario intenta jugar en una casilla ocupada → rechazar el movimiento
- Usuario intenta jugar después de que el juego terminó → mostrar resultado
- La función de victoria se equivoca → revisar todas las líneas (3 filas + 3 columnas + 2 diagonales)
- El tablero se llena sin ganador → empate, no error
- Usuario reinicia el juego → nuevo tablero vacío, X empieza
- La función llama a la IA para su turno → ¿dónde juega la máquina? (algoritmo)

7. Lógica en Pseudocódigo

Antes de JavaScript, escribimos la lógica en palabras claras. Esto es pseudocódigo: un nivel entre el español y el código real.

1. Estructura del tablero:

tablero = [[casilla(vacía), casilla(vacía), casilla(vacía)], [casilla(vacía), casilla(vacía), casilla(vacía)], [casilla(vacía), casilla(vacía), casilla(vacía)]] Cada casilla puede ser: vacía, X, o O

2. Hacer un movimiento:

función hacerMovimiento(fila, columna): SI tablero[fila][columna] no está vacía: devolver "Error: casilla ocupada"
tablero[fila][columna] = turnoActual (X o O) SI hayVictoria(): ganador = turnoActual estado = "terminado" SINO SI
tableroLleno(): estado = "empate" SINO: turnoActual = cambiarTurno() // $X \leftrightarrow O$

3. Detectar victoria:

función hayVictoria(): PARA cada fila: SI los 3 elementos de la fila son iguales Y no están vacíos: devolver TRUE PARA
cada columna: SI los 3 elementos de la columna son iguales Y no están vacíos: devolver TRUE PARA cada diagonal: SI
los 3 elementos de la diagonal son iguales Y no están vacíos: devolver TRUE devolver FALSE

8. Preguntas para Pensar (Crítica)

Pregunta 1: ¿Por qué separamos la lógica del juego de la interfaz? (Pista: ¿qué pasaría si los mezclamos?)

Pregunta 2: ¿Cuál es la diferencia entre 'el usuario intenta jugar en una casilla ocupada' y 'la app no ve esa casilla'? ¿Cómo las maneja el código?

Pregunta 3: Si el tablero está lleno pero no hay victoria, ¿es eso un error? ¿O es un estado válido del juego?

Pregunta 4: ¿Cuántos estados diferentes puede tener el juego en total? (Pista: piensa en todas las combinaciones posibles del tablero)

Pregunta 5: ¿Qué hace más complejo a Tic Tac Toe: su lógica de reglas o la de detectar victoria?

9. Mini-Reto para Esta Sesión (45 min)

Objetivo: Traducir el Tic Tac Toe a pseudocódigo detallado. **Instrucciones:** 1. Dibuja el tablero como una tabla (papel o digital) 2. Escribe las 3 funciones clave en pseudocódigo: • `iniciarJuego()` • `hacerMovimiento(fila, col)` • `hayVictoria()` 3. Para cada función, escribe qué datos necesita y qué devuelve 4. Identifica al menos 3 casos borde y cómo tu lógica los maneja **No escribas código JavaScript.** Escribe en español claro y lógica que alguien sin programación pueda entender. **Tiempo:** 45 minutos **Entrega:** Un documento (papel o digital) con tu diseño **Revisión:** 15 minutos: comentamos tu lógica juntas

10. Qué NO Hacer (Errores Comunes)

■ Mezclar lógica del juego con UI

Malo: guardar el estado en los elementos HTML. Bueno: guardar en un objeto, actualizar HTML desde allí.

■ Escribir código antes de entender el problema

Malo: abrir VS Code e improvisar. Bueno: primero, dibujar y escribir pseudocódigo.

■ Ignorar casos borde

Malo: asumir que todo va bien. Bueno: preguntar 'qué pasa si...?' antes de programar.

■ Dejar la lógica de victoria sin revisar

Malo: solo revisar filas. Bueno: filas + columnas + diagonales.

■ Usar variables sueltas para el estado

Malo: let tablero1, let tablero2, let ganador... Bueno: let estadoJuego = { tablero: [...], ganador: null, ... }

11. Resumen: Idea que Te Llevas

El Tic Tac Toe es un sistema: • Tiene estado (tablero, turno, ganador) • Recibe entrada (movimientos del usuario) • Aplica reglas (validación, victoria, turno) • Produce salida (nuevo tablero, resultado) **Antes de código:** • Entiende todas las reglas • Dibuja el estado • Escribe pseudocódigo • Prevé casos borde **Después, cuando programes:** • La lógica ya está clara • Sabes exactamente qué necesita el JavaScript • Los bugs son menos sorpresas **Lección grande:** Los mejores ingenieros pasan más tiempo diseñando que programando. Tu trabajo hoy es diseñar. La próxima sesión, implementamos.

■ Recordatorio Final

"No salir nunca con más código, sino con una idea clara." Hoy no escribes JavaScript. Hoy entiendes el Tic Tac Toe a nivel de ingeniera. Eso es más valioso.

